MODULE 15

Testing & Optimization

JUnit, Espresso, Profiling, Battery, and Crash Diagnostics

Phase 03 | Android Development with Kotlin

Testing Strategy and Local Unit Tests

- Apply the test pyramid to decide which kind of test each piece of the PNC Mobile app needs
- Choose between fakes, stubs, and mocks and justify the choice
- Write deterministic JUnit tests using Arrange, Act, Assert with behavior-based names
- Cover edge cases with parameterized tests and test data builders



©Logical Imagination Group LLC

The Test Pyramid for Mobile

- Local unit tests run on the JVM in milliseconds: the wide base
- Integration tests combine real components such as Room plus a repository
- UI and end-to-end tests run on a device or emulator: few, slow, valuable
- Source sets: src/test runs locally, src/androidTest runs on a device

Layer	Test Type	Source set	Speed
Mappers, Validators	Local unit	src/test	ms
UseCases, ViewModels	Local unit + fakes	src/text	ms
Repository + Room	Integration	src/androidTest	100s of ms
Retrofit + interceptors	Integration	src/test (MockWebServer)	ms
Screens (Compose)	UI Test	src/androidTest	seconds
Login to transfer journey	End to end	src/androidTest	10s of seconds



©Logical Imagination Group LLC

Anatomy of a Good Unit Test

Arrange, Act, Assert in three visible blocks

Name tests after behavior: returnsErrorWhenAmountExceedsBalance

One behavior per test; a failure should point to one cause

Deterministic: no real clock, network, randomness, or shared state



©Logical Imagination Group LLC

Anatomy of a Good Unit Test (code)

```
class MoneyTest {
    @Test
    fun addingTwoAmountsSumsCentsExactly() {
        val tenCents = Money(10)
        val twentyCents = Money(20)
        val total = tenCents + twentyCents
        assertEquals(Money(30), total)
    }

    @Test
    fun formatsUsingGivenLocale() {
        assertEquals("$1,234.50", Money(123450).format(Locale.US))
    }
}
```



©Logical Imagination Group LLC

Fakes, Stubs, and Mocks

Stub returns canned answers; a fake is a lightweight working implementation

Mock records calls so you can verify interactions

Prefer fakes for repositories: tests survive refactoring

Use MockK to verify a boundary call that has no observable result



©Logical Imagination Group LLC

Fakes, Stubs, and Mocks (code)

```
class FakeAccountsRepository : AccountsRepository {
    private val accounts = MutableStateFlow<List<Account>>(emptyList())
    var refreshResult: ApiResult<Unit> = ApiResult.Success(Unit)
    var refreshCount = 0
    fun emit(list: List<Account>) { accounts.value = list }
    override fun observeAccounts(): Flow<List<Account>> = accounts
    override suspend fun refresh(): ApiResult<Unit> { refreshCount++; return refreshResult }
}

@Test fun sendsAnalyticsOnce() {
    val analytics = mockk<Analytics>(relaxed = true)
    TransferScreenLogic(analytics).onSubmit()
    verify(exactly = 1) { analytics.track("transfer_submitted") }
}
```



©Logical Imagination Group LLC

Parameterized Tests

- Boundary values find most bugs: zero, one, limit, limit plus one
- Parameterized tests run one body against a table of cases
- Provide a companion object inside the test class
  - This object should provide a method that will return a list of parameter sets for the different test runs
  - Those parameter sets will be provided as constructor parameters to the class
  - The @Test function inside the class should use the parameters to execute the code and make the assertion(s)
- Test data builders hide irrelevant fields and show only what matters
- Name each case so a failure identifies the input



©Logical Imagination Group LLC

Edge Cases: Parameterized Tests and Builders

```
@RunWith(Parameterized::class)
class AmountRuleTest(private val amount: Long,
                     private val balance: Long,
                     private val ok: Boolean) {
    companion object {
        @JvmStatic @Parameterized.Parameters(name = "amount={0} balance={1} -> {2}")
        fun cases() = listOf(
            arrayOf(0L, 100L, false), arrayOf(1L, 100L, true),
            arrayOf(100L, 100L, true), arrayOf(101L, 100L, false)
        )
    }
    @Test fun validates() = assertEquals(ok, AmountRule.isValid(amount, balance))
}
```



©Logical Imagination Group LLC

Lesson 1

Skill Check ✓

1. Where does a transfer amount rule belong in the test pyramid?

A) End-to-end UI test
B) Local JVM unit test
C) Manual test only
D) Screenshot test

2. Why prefer a fake repository over a mock in ViewModel tests?

A) Fakes are faster to compile
B) Fakes test outcomes, so tests survive refactoring
C) Mocks cannot return values
D) Fakes need the network

3. Which inputs best find an off-by-one bug in an amount rule?

A) Random numbers
B) Boundary values and their neighbors
C) Only typical values
D) Only negative numbers

4. What makes a unit test non-deterministic?

A) Using assertEquals
B) Reading the real clock or network
C) Using Kotlin
D) Naming it clearly



©Logical Imagination Group LLC